Nox Confidential Token Demo — Passation Projet

Destinataires : Pierre, Abbes **Auteur :** Martin Leclercq **Date :** Mars 2025 **Repo entreprise :** `github.com/iExec-Nox/demo-ctoken`

---

1. Scope du projet

Vision

Application web3 de démo présentant le **protocole de calcul confidentiel Nox** sur Arbitrum Sepolia (testnet). L'utilisateur peut **mint, transférer et auditer des tokens privés (cTokens)** via une UI soignée. L'app doit atteindre la qualité d'onboarding de Zama Portfolio et Inco tout en se différenciant via la **gestion des ACL** et un **outillage développeur embarqué**.

Périmètre fonctionnel

Fonctionnalité	Description	Statut
Landing / Connect Wallet	Page d'accueil + connexion wallet via Reown AppKit	Done
Faucet (modale)	3 tokens (ETH, RLC, USDC) — liens vers faucets externes	Done
Dashboard	Portfolio avec soldes publics + confidentiels, action center	Done
Wrap / Unwrap	Convertir public → confidentiel (et inverse), SDK câblé	Done
Transfer confidentiel	Transfert de cTokens chiffré (encryptInput + proof)	Done
Selective Disclosure / ACL	Donner accès en lecture à un auditeur (addViewer)	Done
Activity Explorer	Historique des transactions (UI faite, données mock)	UI Done
Developer Mode	Affichage code Solidity / SDK dans chaque modale	Done
Light / Dark Theme	Thème clair et sombre avec tokens sémantiques	Done

Dynamic Network Fees	Estimation gas temps réel via wagmi	Done
Responsive	**Adaptation mobile**	**TODO**
Settings	**Page paramètres (thème, wallet, réseau)**	**TODO**
Viewer list	**Lister les viewers ACL actuels**	**TODO**
Subgraph Explorer	**Données réelles dans l'explorer**	**TODO**

Tokens déployés (Arbitrum Sepolia)

Token	Type	Adresse	Decimals
RLC	ERC-20	`0x9923eD3cbd90CD78b910c475f9A731A6e0b8C963`	9
cRLC	ERC-7984	`0x271f46e78f2fe59817854dabde47729ac4935765`	9
NoxCompute	ACL/Viewer	`0x5633472D35E18464CA24Ab974954fB3b1B122eA6`	—

> **Note :** USDC / cUSDC ne sont pas encore déployés. Seul le couple RLC / cRLC est fonctionnel.

---

2. Specs techniques

Stack

Couche	Techno	Version
Framework	Next.js (App Router, Turbopack)	16
Styling	Tailwind CSS + shadcn/ui (New York)	v4
Web3	wagmi + viem + Reown AppKit	wagmi v2, viem v2
SDK confidentiel	@iexec-nox/handle	0.1.0-beta
State	React Context (providers par modale)	—
Queries	@tanstack/react-query	—
Theme	next-themes	—

Wallet	MetaMask, Rabby, Coinbase Wallet, WalletConnect	—
Chain	Arbitrum Sepolia (chainId: 421614)	—
Prix	CoinGecko API (via Next.js API route, revalidate 60s)	—

Variables d'environnement

```
NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID=<reown_project_id>
NEXT_PUBLIC_ARBITRUM_SEPOLIA_RPC=https://arbitrum-sepolia.gateway.tenderly.
```

> RPC public Tenderly — pas de clé API nécessaire. Suffisant pour le testnet.

Commandes

```
npm install       # Installer les dépendances
npm run dev       # Serveur de dev (localhost:3000)
npm run build     # Build production
npm run lint      # ESLint
```

---

3. Overview fonctionnel

Parcours utilisateur

```
Landing Page  ──→  Connect Wallet (Reown AppKit)
                        │
                        ▼
                   Dashboard
                   ├── Portfolio (soldes publics + confidentiels)
                   ├── Action Center
                   │   ├── Wrap    → Modale Wrap/Unwrap
                   │   ├── Unwrap  → Modale Wrap/Unwrap (onglet 2)
                   │   ├── Transfer → Modale Transfer
                   │   └── Share   → Modale Selective Disclosure
                   ├── Faucet (modale) → liens externes
                   └── Explorer → Historique transactions
```

Détail des opérations

Wrap (public → confidentiel) : L'utilisateur envoie des tokens publics (ex: RLC) vers un smart contract qui les bloque. En retour, il reçoit des tokens confidentiels (cRLC) au ratio 1:1.

Unwrap (confidentiel → public) : Opération inverse — brûler les cTokens pour récupérer les tokens publics. Processus en 2 étapes avec un handle intermédiaire.

Transfert confidentiel : Transfert de cTokens entre adresses. Le montant est chiffré côté client avant envoi au contrat.

Selective Disclosure (ACL) : Donner accès en lecture du solde à un tiers (auditeur, régulateur). Fonctionne par *handle* — doit être re-fait après chaque transaction qui modifie le solde.

Developer Mode : Activé par défaut. Affiche le code Solidity/SDK appelé dans chaque modale. Toggle global dans le header.

---

4. Architecture

Structure des dossiers

```
/app
  /(landing)/           # Landing + Terms → Layout: Header + Footer
  /(app)/               # Dashboard + Explorer → Layout: Topbar + DashboardHeac
  /api/prices/          # Proxy CoinGecko
/components
  /layout/              # Topbar, Header, Footer, DashboardHeader, MobileMenu
  /landing/             # HeroSection, FeatureCard, FeaturesSection
  /dashboard/           # DashboardContent, Assets, ActionCenter, TokenRow...
  /modals/              # FaucetModal, WrapModal, TransferModal, SelectiveDisc
  /shared/              # CodeSection, InfoCard, ErrorMessage, TxStatus, Logo..
  /explorer/            # ExplorerContent, ActivityTable
  /ui/                  # Primitives shadcn (ne pas modifier)
  providers.tsx         # Hiérarchie des providers
/hooks/                 # Hooks de transaction, balances, UI
/lib/                   # Contrats, tokens, wagmi, gas, format, utils, ABIs
/docs/decisions/        # ADR-0001 à ADR-0012
```

Hiérarchie des providers

```
ThemeProvider (next-themes)
  └── WagmiProvider (Reown AppKit + wagmi)
        └── QueryClientProvider (@tanstack/react-query)
              └── TooltipProvider
                    └── AppKitThemeSync
```

```
                    └── FaucetModalProvider
                         └── WrapModalProvider
                              └── TransferModalProvider
                                   └── SelectiveDisclosureModalProvider
                                        └── {children}
```

Pattern Modal (Provider + Modal + Hook)

Toutes les modales suivent le même pattern :

1. **Provider** (`xxx-modal-provider.tsx`) — React Context + state `open/setOpen` + méthode `openXxx()`
2. **Modal** (`xxx-modal.tsx`) — UI Dialog shadcn + logique formulaire
3. **Hook** (`useXxxModal()`) — Exposé par le provider pour ouvrir la modale depuis n'importe où
4. **Câblage** — Le Provider monte le composant Modal et s'insère dans `providers.tsx`

Pattern Hook de transaction

```
// Contrat commun à tous les hooks tx
type Step = "idle" | "approving" | "wrapping" | ... | "confirmed" | "error"

interface UseXxxResult {
  step: Step;           // État courant de la transaction
  error: string | null; // Message d'erreur formaté
  txHash?: `0x${string}`;
  action: (...args) => Promise<void>;  // Déclencher la transaction
  reset: () => void;    // Retour à l'état initial
}
```

Hooks existants : `useWrap` , `useUnwrap` , `useConfidentialTransfer` , `useAddViewer` , `useDecryptBalance` , `useEstimatedFee` , `useDevMode` , `useTokenBalances` , `useConfidentialBalances`

Source unique de vérité pour les tokens

`lib/tokens.ts` est le **seul fichier** définissant les tokens. Les tokens confidentiels sont dérivés automatiquement :

```
// Ajouter wrappable: true + confidentialAddress au token de base
// → cRLC est créé automatiquement dans confidentialTokens
```

```
{ symbol: "RLC", wrappable: true, confidentialAddress: "0x271f..." }
// Génère → { symbol: "cRLC", address: "0x271f...", underlying: "0x9923..."
```

---

5. Choix techniques et ADRs

Le projet documente ses décisions architecturales dans `/docs/decisions/` :

ADR	Décision	Justification
ADR-0001	Reown AppKit (pas RainbowKit)	Meilleure intégration WalletConnect, support multi-wallet natif
ADR-0003	Faucet = modale (pas une page)	Accessible depuis plusieurs points d'entrée, pas de navigation
ADR-0004	Forcer Arbitrum Sepolia	Pas de hook custom, utiliser les mécanismes natifs AppKit + MetaMask
ADR-0005	Wrap/Unwrap = 2 onglets	Un seul composant modale, approve exact (pas infinite)
ADR-0010	Fees dynamiques	Hook `useEstimatedFee` basé sur gasPrice × gasLimit, refresh par bloc
ADR-0011	Dev Mode activé par défaut	Différenciateur clé — premier visiteur voit immédiatement le code
ADR-0012	Réorg composants par thème	Sous-dossiers thématiques (layout, landing, dashboard, modals, shared)

---

6. Smart contracts — Flows détaillés

Wrap (RLC → cRLC)

```
1. RLC.approve(cRLC_address, exact_amount)   ← ERC-20 classique
2. cRLC.wrap(user_address, amount)            ← Montant en clair (uint256)
```

> **Pas d'encryptInput** pour wrap — le montant est en clair.

Unwrap (cRLC → RLC) — 2 étapes

```
1. encryptInput(amount)  → { handle, inputProof }     ← Client-side (Web Crypto
2. cRLC.unwrap(user, user, handle, inputProof)         ← Retourne un AUTRE hand
3. Décoder event "UnwrapRequested" → unwrapHandle      ← Différent du handle in
4. cRLC.finalizeUnwrap(unwrapHandle, amount, "0x00")   ← Proof mocké pour l'ins
```

> **Attention :** Le handle retourné par `unwrap()` est différent du handle input. Il faut décoder l'event pour récupérer le bon handle de finalization.

Transfert confidentiel

```
1. encryptInput(amount)  → { handle, inputProof }
2. cRLC.confidentialTransfer(recipient, handle, inputProof)
```

Selective Disclosure (ACL)

```
1. confidentialBalanceOf(user)  → balance_handle (bytes32)
2. NoxCompute.addViewer(balance_handle, viewer_address)
```

> **Per-handle, pas per-token.** L'autorisation est liée au handle de balance, pas au token. Dès qu'une transaction modifie le solde (wrap, unwrap, transfer), le handle change et l'autorisation est automatiquement perdue. **Il n'y a donc pas besoin de mécanisme de révocation** — l'expiration est implicite. Si l'accès doit persister, il faut re-appeler `addViewer` avec le nouveau handle après chaque opération.

Points critiques

- **Cooldown 2-3s** entre les étapes séquentielles (NoxCompute/TEE a un délai)
- **Gas overrides** obligatoires sur Arbitrum (MetaMask sous-estime)
- **HandleClient** = browser only (Web Crypto API, jamais côté serveur)
- **Approve exact** uniquement (bonne pratique web3, pas d'infinite approval)
- **Toujours** `waitForTransactionReceipt` après `writeContractAsync`
- **Handles éphémères** — relire `confidentialBalanceOf` après chaque opération

---

7. Design System

Tokens CSS sémantiques

Les couleurs sont définies en CSS variables dans `globals.css` (`:root` pour light, `.dark` pour dark). **Règle absolue : jamais de couleur hardcodée** (`text-white` , `bg-[#...]`), toujours un token sémantique.

Token	Light	Dark	Usage
`bg-background`	`#dde2ee`	`#0f1119`	Fond de page
`bg-surface`	`#edf0f8`	`rgba(255,255,255,0.03)`	Cards, conteneurs
`bg-primary`	`#748eff`	`#748eff`	Boutons principaux, accents
`text-text-heading`	`#111827`	`#ffffff`	Titres
`text-text-body`	`#6b7280`	`#94a3b8`	Texte courant

Fonts

Font	Poids	Usage
Anybody	700	Titres hero (72px), titres de section (32px)
Mulish	400, 500, 700	UI (topbar, boutons, navigation)
Inter	400, 500, 700	Contenu des cards

Conventions composants

- **Nommage fichiers :** `kebab-case.tsx`
- **Pas de** `export default` — uniquement des named exports
- **Single responsibility** — extraire en sous-composants quand un pattern se répète
- **Données en dehors du JSX** — extraire les arrays en `const` au niveau module
- **Accessibilité** — `aria-label` sur les boutons icônes, sémantique HTML, navigation clavier

8. Outils de dev

Developer Mode (intégré à l'app)

Activé par défaut (`useDevMode()` , persisté en localStorage). Affiche dans chaque modale :

- Le code Solidity de la fonction appelée
- Le code SDK correspondant
- Bouton copier

Toggle global dans le DashboardHeader et le menu mobile.

Architecture Decision Records (ADR)

12 ADRs dans `/docs/decisions/` documentant chaque choix architectural significatif.

Fichiers de référence

Fichier	Contenu
`CLAUDE.md` (racine)	Specs complètes du projet, patterns, priorités
`components/CLAUDE.md`	Design system, conventions composants, inventaire
`lib/tokens.ts`	Configuration unique de tous les tokens
`lib/contracts.ts`	Adresses des contrats
`lib/confidential-token-abi.ts`	ABI du token confidentiel (cRLC)
`lib/nox-compute-abi.ts`	ABI NoxCompute (ACL/Viewer)

---

9. Todo : Pierre, Abbes & Martin

Contributions & maintenance

Responsabilité	Détail	Attribution
Review de code	Review des PRs sur `iExec-Nox/demo-ctoken`	Pierre & Abbes
Issues	Remonter les bugs et suggestions via GitHub Issues	Pierre & Abbes

Contributions	Implémenter les features TODO (responsive, settings, activity, view page)	Martin
Maintenance	Mettre à jour les dépendances, corriger les bugs	Martin
Subgraph	Intégrer les données réelles dans l'Activity Explorer quand le subgraph est prêt	Martin

Review & support

Responsabilité	Détail	Attribution
Avis technique	Valider les choix d'architecture	Pierre & Abbes
Review	Relire le code et signaler les points d'amélioration	Pierre & Abbes
Support	Répondre aux questions quand Martin itère	Pierre & Abbes

Notes

- Le repo entreprise est sur `github.com/iExec-Nox/demo-ctoken` (branche `main`)
- Le code ne contient **aucun fichier lié à Claude** (filtré par `.gitignore` sur la branche enterprise)
- Les ADRs et `CLAUDE.md` servent de documentation — ils sont lisibles indépendamment de l'outil

---

10. Workflow Git proposé

Situation actuelle

Le repo entreprise (`iExec-Nox/demo-ctoken`) est une copie de la branche de travail, sans les fichiers de tooling Claude. La branche `main` est la seule branche active.

Workflow recommandé pour contributions

```
main (protégée)
  └── feature/xxx    ← Branche par feature / fix
      └── PR → Review → Merge
```

Conventions de commit : Conventional Commits (`type(scope): description`)

Type	Usage

`feat`	Nouvelle fonctionnalité
`fix`	Correction de bug
`style`	CSS, design, mise en forme
`refactor`	Restructuration sans changement fonctionnel
`docs`	Documentation
`chore`	Config, dépendances, tooling

Exemple : `feat(settings): add theme toggle page`

Setup local

```
git clone git@github.com:iExec-Nox/demo-ctoken.git
cd demo-ctoken
npm install
cp .env.example .env.local   # Renseigner WALLETCONNECT_PROJECT_ID
npm run dev
```

---

11. Pièges connus (lessons learned)

Piège	Explication
Doc SDK ≠ contrat déployé	La doc Nox mentionne `mint` / `burn`, mais le contrat utilise `wrap` / `unwrap` / `finalizeUnwrap`. Toujours vérifier sur Arbiscan.
Handles éphémères	Le handle de balance change après **chaque** transaction. Toujours re-lire `confidentialBalanceOf` après une opération.
Unwrap : 2 handles différents	Le handle passé à `unwrap()` ≠ celui retourné par l'event. Décoder `UnwrapRequested` pour le bon handle de finalization.
encryptInput vs clair	`wrap()` prend un montant en clair. `unwrap()` et `transfer()` nécessitent `encryptInput`. Ne pas généraliser.
HandleClient = browser only	Utilise `crypto.subtle` (Web Crypto API). Ne jamais importer côté serveur / SSR.

Approve exact, pas infinite	Bonne pratique web3 — approve uniquement le montant exact nécessaire pour chaque opération.
Cooldown inter-étapes	2-3 secondes entre les tx séquentielles (TEE/NoxCompute a un délai de traitement).
Gas Arbitrum	MetaMask sous-estime le gas. Utiliser `estimateGasOverrides()` avec buffer 20%.
ACL per-handle, pas de revoke	`addViewer` est lié au handle, pas au token. Dès que le handle change (après toute tx), l'autorisation est perdue automatiquement. Pas besoin de revoke — c'est *by design*.

---

12. Prochaines étapes

Priorité haute (EthCC)

1. **Responsive mode** — Adaptation mobile complète
2. **Settings page** — Toggle thème, infos wallet, config réseau
3. **Activity page** — Page `/explorer` avec données réelles (subgraph) au lieu des données mock

Priorité moyenne

4. **View page** — Page permettant à un viewer autorisé (via ACL) de consulter les éléments qui lui ont été partagés (soldes, handles autorisés)
5. **Viewer list** — Afficher la liste des viewers ACL ayant accès au handle courant (pas de revoke nécessaire : le changement de handle invalide automatiquement les accès)
6. **Subgraph Activity Explorer** — Remplacer les données mock par des données on-chain réelles
7. **USDC / cUSDC** — Intégrer quand les contrats sont déployés (juste ajouter l'adresse dans `lib/tokens.ts`)

Nice to have

8. **On-chain faucet** — Remplacer les liens externes par un contrat faucet
9. **Page Developer Tools** — Page dédiée `/developer` avec snippets et documentation
10. **Animations** — Transitions entre les étapes des modales

---

Document généré le 10 mars 2025 — Nox Confidential Token Demo v1.0